

Python

Python is a very popular Prog. Lang.

Created by Guido Van Rossum, & released in 1991 at CWT (Centrum Wiskunde & Informatica) .Netherland.

General purpose High level prog. language.

Dynamic. (No need to mention it before like $a=10 \rightarrow$ type int (shown).
(Use anywhere)

Why Python? -
Simple & Easy to Learn. (Simple Syntax) $\rightarrow$ Only salt find out
Platform Independent (Works on Different Platforms)
Free & open Source.

Interpreted (bytecode, Compiled) (By default $\rightarrow$ Interpret line by line
 $\uparrow$ speed.)

Rich library Support. (1440 this lib present) Like Maths AI, NUMPY.

Embeddable & Extensible. (C to Pyb change code)

Portable (Machine to Another Machine)

Robust (Exc. Handle, only Salt oriented)

Where Python is Used -

SUI Based Desktop Appln

Graphic design, image processing applications, Games &

Scientific / Computational Appln

ML, AI, Neural Networks

Data Science, Data visualization

IoT (Internet of Things).

Database Development

Web Frameworks & Appln

SIW Development

Mathematics

System Scripting

At Run time take input from User

num 1 = input("enter first number")

Python - Some Minus Point / Drawback :-

Python is very powerful yet simple language with so many advantages. it is not the perfect prog. language.

1.) Not the faster language :-

- Python is an interpreted language not a fully compiled one.
- Python is first semi-compiled into an internal byte code, which is then executed by a python interpreter.
- So here python is little weaker though it offers faster development times but execution - Times are not that fast compared to some compiled language.

2.) Lesser Libraries than C, Java, Perl -

Python offers library support for all computing program but its library is still not competent with language like C, Java, Perl as they have larger collections available.

3.) Not strong on Type-binding -

Python interpreter is not very strong on type mismatch issues.

For example, if you declare a variable as integer but later store a string value in it.

Python won't complain or pin-point it.

Working of Python :-

Before we need to start working in Python, we need to install Python on our computer. There are multiple Python distributions available today.

Print ("Hello, world!") → O/P → Hello, world!

>>> Print ("Hello, world!") → Hello, world

Python Indentation:-

Indentation refers to the spaces at the beginning of a code line.

In Python Indentation is very important.

Strongly Dynamically Typed.

Variable In Python:-

↓
like a container

Use to store data so that manage it efficiently.

a = 10

a = Hello

At Run time it automatically find out type of variable.

Declare the Variable :-

First letter or underscore

__Var ✓
_Var ✓

Ivar ✓
Ivar x

Var ✓

VAR, Var (Diff Mean).

Var 1 = 5
Print (Var1)

O/P - 5

__Var1 = 5
Print (__Var1)
5

IVar1 = 5
Print (IVar1)

Syntax Error - Invalid

Var 1 = 5
Print (Var 1)

Synt Error

Var - Var2 = 5
Print (Var - Var2)

5

def = 5
Print (def)

↓
Syntax Error - def key

a = 10

A = 20

Print (a) → O/P → 10

Print (A) → O/P → 20

Data types in Python:-

1. Numeric Type:-

int
float
Complex (Imaginary)
Rarely use

2) Text Type:-

• str

[widely Used]
Name & Add

3. Boolean Type

bool (T & F)

4) Sequence Type:-

list (Mutable)
Tuple (Immutable)
range

5) Mapping Type:

• dict → key value pair
dictionary → No seq (key value pair)
data structure

6) Set Type :-

• Set (Unordered data but Unique).

7) Binary Type.

byte (image, text, video, etc)
bytearray

8) None Type

Special type of data type
(Uninitialized, Unknown data).

Ex age = 32
type (age)
→ int

weight = 72.58
type (weight)
→ float

is-boy = True
type (is-boy)
bool

list 1 = [100, 200, 300]
print (list 1)

⇒ [100, 200, 300] → sq bracket
type (list 1)

⇒ list

tuple
tup = (10, 20, 30)

dict 1 = {'name': 'Varun', 'age': 32,
'city': 'Chandigarh'}

type (dict 1) ⇒ dict

print dict
print (dict 1 ['city'])
Chandigarh

set
set 1 = {1, 2, 3, 3, 3, 4} byte & bytearray
byte = bytes ([65, 66, 67])
(High. ALU)

Complex
com = 1 + 2j

range
num = range (15)
print (num)
range (1, 5)

None
no one no = None

bytes & bytearray

byte = bytes ([65, 66, 97])

print (byte)

b'ABa'

byte = bytearray ([65, 66, 97])

bytearray (b'ABa')

⇒ How to take Input from User? -

name = input ("Enter your name: ")

→ print ("Hello, " + name)

Enter your name

Hello, Priya

print (type (name))

<Class 'str'>

Taking input from the user as Integer

num = (input ("Enter a number: ")) → Enter num = 10

print (num) → 10

print (type (num)) → string (class)

By default ^{Type} Return
↓
string

⇒ num = num + 2
print (num)

Error

// Num is string
we add 2 in string which is wrong.

num = int ((input ("Enter a number: ")))

print (num)

print (type (num))

num = num + 2

print (num)

Enter a number : 10

10

<Class 'int'>

12

num = bool ((input ("Enter a number: ")))

print (num)

print (type (num))

Enter a number = 10

True

<Class 'bool'>

→ Enter a number ☐

False

<Class 'bool'>

Default Installation available from ^{www}Python.org is called Python installation and comes with Python interpreter, Python IDLE and pip (package installer)

⇒ There are many Python distributions available these days. Anaconda Python distribution is one such highly recommended distribution that comes preloaded with many packages & libraries (eg Numpy, Scipy, Pandas libraries etc).

⇒ Many Popular IDEs are also available.
eg - Spyder IDE, Pycharm IDE etc.

We can work in Python in following different ways -

- (1) In Interactive mode (also called Immediate Mode)
- (2) In script mode.

Working In Interactive Mode (Python IDLE)
Interactive mode of working means you type the command - one command at a time, and the Python executes the given command there & then gives you output.

⇒ In Interactive Mode, you type the command in front of Python command prompt >>>

>>> 2+5 ← Command / Expression given here.

Result → 7
Returned by
Python.

To work in Interactive mode, follow the Process given below -

- (i) click start button → All programs → Python 3.6.x IDLE (Python GUI)

→ `Print ("Hello")`
`print ('Hello')`] → Hello

→ `"Hello"` → `'Hello'`

→ `>>> s = 13`

`>>> s`

13

→ `>>> s + 20`

33

Working in script mode:-

With interactive mode, you cannot do or save all the commands in the form of program file and can not see all the output lines together, so the solution for this problem is the script mode.

To work in a script mode, we need to do the following

Step 1:-

(i) Create Module / script / Program file

(i) Click Start button → All program → Python 3.6 x-IDE

(ii) Click File → New in IDLE Python Shell.

(ii) In the new window that opens, type the commands you want to save in the form of a program (or script)

`print("Hello World!")`

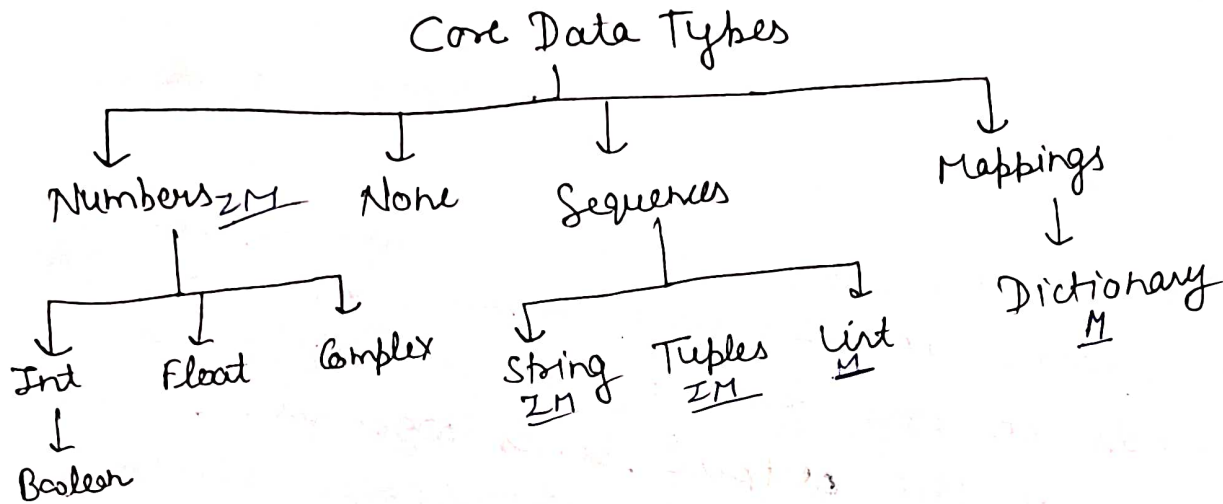
(iv) Click File → Save and then save the file with an extension .py.

Step 2:- Run Module / script / Program file

(i) Open the desired program / script file that we created in previous Step 1 by using IDLE's File → Open...

(ii) Click Run → Run Module command in the open program / script file's window.
We may also press F5 key.

It will execute all the commands stored in module / Program / Script that we had opened and show the complete output in a separate python shell window.



Mutable & Immutable Types :-

The Python data objects can be broadly categorized into two mutable and immutable types in Python ; in simple world changeable or modifiable and non modifiable

The mutable types are those where values can be changed in place

- 1.) List
- 2.) Dictionary
- 3.) Set

Immutable Types —

The immutable types are those that can never change their values in place. In python.

Integer

Floating point numbers

Boolean

Strings

Tuples

The integers:-

The numeric literals in Python can belong to any of the following four different numerical types.

int (signed integers)

Integer literals are whole numbers without any fractional part. The method of writing integer constants has been specified in the following rule -

An integer constant must have at least one digit and must not contain any decimal point. It may contain either (+) or (-) sign.

A number with no sign is assumed to be positive. Commas cannot appear in an integer constant.

1.) Decimal Integer Literals:-

Seq. of digits taken to be decimal integer unless it begins with 0.

For instance - 1234, 41, +97, -17

2.) Octal Integer Literals - 0-7 8, 9 - Not allowed.

3.) Hexadecimal Integer Literals -

0-9

A-F

Floating Point Literals - A Real literal in fraction form consisting of signed & unsigned digits including a decimal point between digits.

It may also have either + or - sign preceding it.

A real constant

>>> 5.0 → 5.0

>>> 5.50 → 5.5

>>> 5.8540000 → 5.854

>>> 5.05 → 5.05

>>> 10/2 → 5.0

>>> 10.0/2 → 5.0

>>> 10.0/2.0 → 5.0

>>> (4+8)/2 → 6.0

>>> -7 → -7

$$>>> (-7+2) * (-4) \rightarrow 20$$

$$>>> (15-1) * 3 \rightarrow$$

$$>>> (1+9) / (-5+5) \rightarrow \text{error} \quad / 22 \div 1.5 = 2$$

$$>>> 3/4 = 0.75$$

$$>>> 9.8765000 \rightarrow 9.8765$$

$$>>> * * \rightarrow \text{exponentiation}$$

$$// \rightarrow \text{Quotient or floor division}$$

$$/ \rightarrow \text{Division}$$

$$\% \rightarrow \text{Remainder}$$

$$>>> 9 * (1/2) \rightarrow 3.0$$

$$>>> (1 + ?) * 2$$

$$>> 20 // 6 \rightarrow 3 \text{ (floor division)}$$

$$5/2 \Rightarrow 2.5$$

$$6/2 \rightarrow 3.0$$

$$5.0/2 \rightarrow 2.5$$

$$6.0/2 \rightarrow 3.0$$

$$5.0/2.0 \Rightarrow 2.5$$

$$10/3 \rightarrow 3.33333$$

$$22/5 \rightarrow 4.4$$

$$5 // 2 \rightarrow 2$$

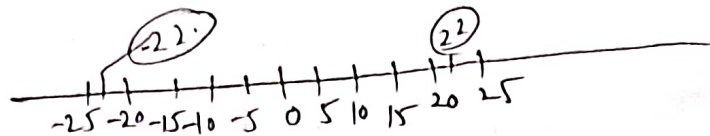
$$6 // 2 \rightarrow 3$$

$$5.0 // 2 \rightarrow 2.0$$

$$10 // 3 \rightarrow 3$$

a, b let a & b two no.

a % b $\rightarrow$ Result $\rightarrow$ if b is +ve $\rightarrow$ Result +ve
if b is -ve $\rightarrow$ Result -ve



if a is +ve $\rightarrow$ left
if a is -ve $\rightarrow$ Right

$$22 \div 1.5 \rightarrow 2$$

$$22 \div 1.5 = -3$$

$$-22 \div 1.5 = 3$$

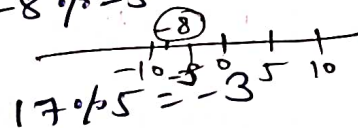
$$-22 \div 1.5 = -2$$

$$8 \div 1.5 = 3$$

$$-8 \div 1.5 = +2$$

$$8 \div 1.5 = -2$$

$$-8 \div 1.5 = -3$$



$$17 \div 1.5 = -3$$

$$17 \div 1.5 = -2$$

$$17 \div 1.5 = -3$$

String :- A String data type that holds string data
Hold any type of known character i.e letter, No,
Special character of any known scripted language.

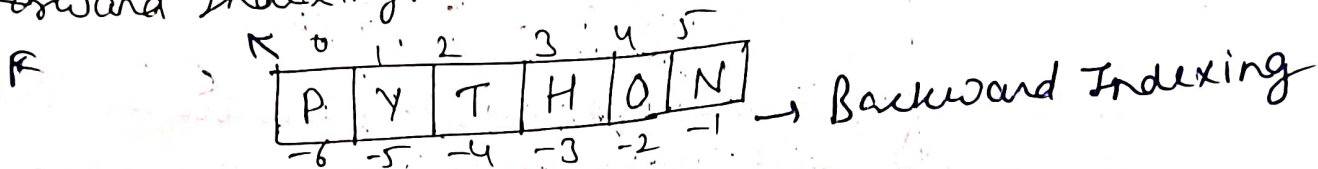
Legal string in Python—

"abcd", "1234", "£% ^ \$" , '????' .

String as a sequence of character sequence

String as a sequence of character—
A Python string is a sequence of character & each character can be individually accessed using its index.

Forward Indexing:-



name[0] = 'P' = name[-6]
name[1] = 'Y' = name[-5]
name[2] = 'T' = name[-4]
name[3] = 'H' = name[-3]
name[4] = 'O' = name[-2]
name[5] = 'N' = name[-1]

name[5] = 'N' = name[-1]

Since length of string variable can be determining using function `len (<string>)`.

the string is at index —

function `len (<string>)`:
= first character of the string is at index —

Since length of string variable can be determined using `len()` (string), we can say that

[illegible]

Second last character of the string is at index $(\text{length}-2)-2$

last character of the string is at index $(\text{length}-1)$ or $s-1$

0-9-1 length N9ccen - Error

Python character set :-

Letters - A-Z, a-z

Digits - 0-9

Special Symbols → Space + - * / % \ () [] { } | ! = < > ' " ; : % ! & # < > = @ (Underscore).

White spaces → Blank space, tabs (→), Carriage return (↵), newline, formatted.

Other characters → Python can process all ASCII & wide char as per of data or literals.

Tokens - Individual words & punctuation marks are called Tokens or lexical unit or lexical elements. The smallest individual unit in a prog is known as token or lexical unit.

Keywords :- 32 + 3 New (False, None, True)

Keywords are the words that convey a special meaning to the language compiler / interpreter. They are reserved for special purpose and must not be used as normal identifier names.

Python contains the following keywords.

False	assert	del	for	in	or	while
None	break	elif	from	is	pass	with
True	class	else	global	lambda	raise	yield
and	continue	except	if	nonlocal	return	
as	def	finally	import	not	try	

Value taken at runtime by user -

```
num1 = input("enter first number")
```

```
num2 = input("enter second number")
```

```
print(num1 + num2)
```

enter first number 12

enter second number 4

// Consider string value

O/P → 124

NOTE Input fn by default return string value.

Type Casting :- change one data type into another data type.

1.) Implicit type casting :- Python (itself)

2.) Explicit type casting :- User interfere.

```
num1 = int(input("enter first number"))  
num2 = int(input("enter second number"))  
print(num1 + num2)
```

124

Enter first number 12
Enter second 4 14

26

```
num1 = 20  
num2 = 20.5  
print(num1 + num2)  
print(num1 * num2)
```

40.5
400.0

// Type Casting

Implicit → lower to higher
int to float

Explicit → higher to lower
float to int
Date conversion

```
a = 11  
f = float(a)  
print(f)
```

```
b = hex(a)  
print(b)  
c = oct(a)  
print(c)
```

```
d = bin(a)  
print(d)  
char = 'A'  
e = ord(char)  
print(e)  
e = 65 (unicode)
```

```
char = chr(e)  
print(char) - A
```

11/0
0x6
0013
001011
65
A

Escape character:-

These are special characters used to represent non printable special characters within a string.

\: Backslash

\n: Newline

\t: Tab

\r: Carriage Return

Print('I'll call you later') → Error

Print('I will call you later') → I will call you later. ✓

Print("I'll call you later")

I'll call you later. ✓

→ Print('I'll call you later') (\ → Escape Character Backslash

I'll call you later.

→ Print("I'll call you later")

I'll call you later.

→ Print("Prepare well for 'your academic' exam")

Syntax Error → 2 times double quotes

→ Print("Prepare well for 'your academic' exam")

Prepare well for "your academic" exam.

→ Print("Prepare well for \"Gate-2024\" exam")

Prepare well for "your academic" exam.

a = Hello, World!
Print(a)

a = "Hello, \n World!"
Print(a)

Hello,
World!

a = Hello, \t World!
Print(a)
Hello, World!

\t → Show Space

→ a = "Hello, \t\t\t world!"
 print(a)
 Hello, world!

a = "no - benefits - of - study \x Huge - benefits"
 print(x)

Huge - benefits - of - study

Operator Precedence :-

()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, isnot, in, not in	Comparison, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR
=, +=, -=, /=, *=	Assignment operators

$2^2 = 4$

$2^2 = 4$

$2^2 = 4$

F&F
F
T

F&F
F
F

$a = 2$

F&F=F

Associativity:

$$2 * 3 * 2$$

$$\begin{array}{l} 2 * 3 = 6 \\ 6 * 2 = 12 \end{array}$$

Exponent $\rightarrow R$ to L

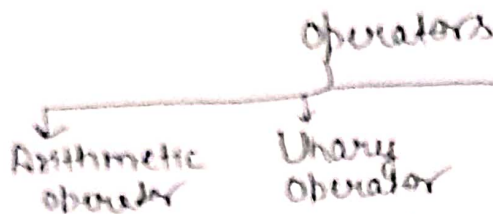
Assignment operators $\rightarrow R$ to L

$$a = b + c \quad a = 5, b = 2, c = 3$$

$$a = b + c$$

$$b = b + c$$

$$b = b + c \quad (2 + 3) = 5$$



1) Arithmetic operators :- Perform basic arithmetic operations. Such as

Addition

Subtraction

Multiplication

Division // Floor division $\rightarrow 15/4 = 3.75$ $15//4 \Rightarrow 3$ (lower / floor value / chopped decimal)

Modulus

Exponentiation. $2 ** 3 \rightarrow 2^3 = 8$

$$10 \% 2 = 0 \quad [\text{show Remainder}]$$

$$10 \% 3 = 1$$

$$10 \% 1 = 0$$

$$10 \% 0 = \text{Infinite value / error}$$

Precedence :- $** > * , / , \% > + , -$

R to L L to R L to R

$$\rightarrow 2 * 4 // 3 * 2 + 3 - 5$$

$$\begin{array}{l} 2 * 4 = 8 \\ 8 // 3 = 2 \\ 2 * 2 = 4 \\ 4 + 3 = 7 \\ 7 - 5 = 2 \end{array}$$

2) Unary operators :- The operators unary '+' precedes an operand

If $a = -4$ then $+a$ means 4

This operator reverse the sign of the operand value.

If $a = 5$ then $+a$ means 5

If $a = 0$ then $+a$ means 0

If $a = -5$ then $+a$ means 5

Logical operator :-

$$b + c$$

$$b = b + c$$

$$a = 10$$

- In Python, logical operators are used to combine multiple conditions or expression & produce a boolean result
- There are three logical operators in Python.

and :- It returns true if both the operands are true, otherwise it returns false.

AND :- It returns true if both the operands are true, otherwise it return false.

NOT :- It returns the opposite Boolean value of the operand, if the operand is true, it returns false, and if the operand is false, it returns True.

Program :-

```
age = 15
income = 20000
if age >= 18 or income >= 15000:
    print("eligible for the loan")
else:
    print("not eligible")
```

If $15 > 18$ - NO
F
the st: $15 > 18$ and $20000 > 15000$
F & T
F → Result
else part ✓

a = T
b = F
if not a: ✓ F ✓
print("my name is a") # th
will not be executed
if not b:
print("my name is b") # other:
my name is b

NOT operator — not — logical not
Reverse the result i.e relation false if result is true

not 0 → True not (5 < 10) = false
not 5 → false not (5 > 10) = True

Identity operator :- compare two object whether they point to the same memory location.

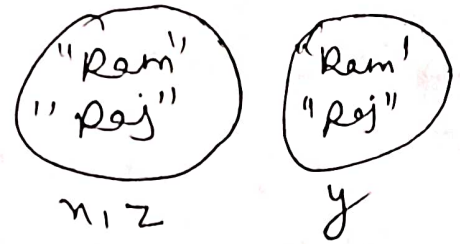
is
is not

These compare not only the values, but also the memory location refers to the object.

1) Is $\rightarrow$ Return true if both objects are same & point to same memory location.

2) Is Not $\rightarrow$ Return true if both objects are not same

```
x = ["Ram", "Raj"]  
y = ["Ram", "Raj"]
```



```
Print[x is y]
```

False $\Rightarrow$ value & location

```
Print(x == y)
```

True $\Rightarrow$ only values

```
z = x  
True
```

```
print(x is not y)  
True (values & location)  
print(x != y)  
False (values)
```

Membership operator:-

In :- Whether variable in sequence

Not In :- Whether variable not in sequence

Punctuators :-

Punctuators are symbols that are used in programming languages to organize sentence structures, and indicates the system & emphasis of expressions, statements and program structure.

Most Common Punctuators of Python Programming language are-

' " # \ () [] { } @ , : ; . ' =

Python Program:-

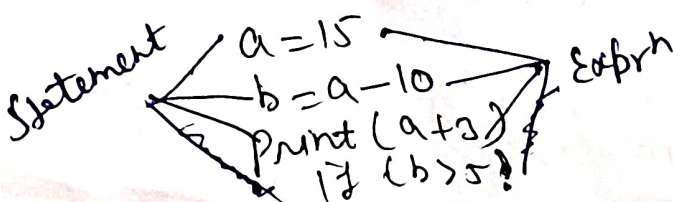
This prog shows a prog components

Comments begin with #.

```
def SeeYou():
```

$\leftarrow$ function

```
Print("Time to say Good Bye!!")
```

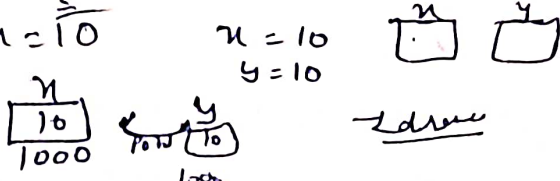


Equality (==) and Identity (is) two things are identical or not - & share same mem location or not. (T & F case).

Print(a, b)
235 235

a is b
True

a == b
True

$n = \frac{5}{10}$ $n = 10$ $y = 10$

True

Not create new mem location. Memory reuse.

$n = 10$
 $y = 20$

Print(n is y) → F
Print(n is 2) → T
Print(y is 2) → F

Comparison operators :-

In Python, Comparison operators are used to compare two values and return a Boolean value (T or F) depending on the result of the comparison.

== Equal to :- Return true if the values on either side of the operator are equal, and false otherwise.

!= Not equal to :- Return true if the values on either side of the operator are not equal, and false otherwise.

> greater than :- Return true if the value on the left is greater than the value on the right and false otherwise.

< less than :- Return true if the value on the left is less than the value on the right and false otherwise.

>= greater than or equal to :- Return true if the value on the left is greater than or equal to the value on the right and false otherwise.

<= less than or equal to :- Return true if the value on the left is less than or equal to the value on the right, and false otherwise.

Ex:- $1 == 2$
F

$1 == 2$
F

$1 != 2$
T

$1 != 1$
F

$10 > 5$
T

$10 < 5$
F

$5 > 5$
F

$< > <= >=$
L-R

$= = ! =$
L-R

Print (4 > 6 == false)
 $7 == 7$
 T ✓

$T = \frac{LR}{J} = T = 7$
 $T = 7$
 F ✓

00
 01
 10
 11

Bitwise operator in Python:-

In Python, bitwise operators are used to perform operations on individual bits of binary numbers.

There are 6 bitwise operators.

& (Bitwise AND)

| (Bitwise OR)

~ (Bitwise NOT)

^ (Bitwise XOR)

<< (Bitwise left shift)

>> (Bitwise Right shift)

0b
 01
 02

Precedence: ~ (highest) <<>>, &, ^, | (lowest)

Bitwise AND

a = 0b1010

b = 0b1100

c = a & b

print(bin(c))

O/P → 0b1000

$\begin{array}{r} 1010 \\ 1100 \\ \hline 1000 \end{array}$

Bitwise OR

a = 0b1010

b = 0b1100

c = a | b

print(bin(c))

O/P → 0b1110

Bitwise Left shift

a = 0b1010

c = a << 2

print(bin(c))

O/P

0b101000

$\begin{array}{r} 1010 \\ 1010 \\ \hline 101000 \end{array}$

1010 → 10
 10100 → 20
 101000 → 40

Bitwise NOT

a = 0b1010

c = ~a

print(bin(c))

Output =

-0b1011

1

(signed)
 By default
 0101
 0101
 ~0101
 1010

neg no
 0101
 1010
 1111

Bitwise Right shift

a = 0b1010

c = a >> 1

print(bin(c))

O/P

0b101

$\begin{array}{r} 1010 \\ 1010 \\ \hline 1010 \end{array}$

1010 → 10

0101 → 5

01010 →

Bitwise XOR

a = 0b1010

b = 0b1100

c = a ^ b

print(bin(c))

O/P 0b0110

$\begin{array}{r} 1010 \\ 1100 \\ \hline 0110 \end{array}$

$\begin{array}{r} 1010 \\ 0101 \\ \hline 1111 \end{array}$

$\begin{array}{r} 1010 \\ 1010 \\ \hline 1010 \end{array}$

101 2 100 | 011 (Use Precedence here).

100 / 011

$$\begin{array}{r} 101 \\ 100 \\ \hline 100 \end{array}$$
$$O(p) \rightarrow \quad \quad \quad 111$$
$$\begin{array}{r} 1 \\ 7 \overline{) 72} \\ \underline{72} \\ 0 \end{array}$$

50

No / string / literals as operand :-

No / string / literals as operand :- $un = [1, 2, 3, 4, 5]$
when and operator has its own operands as no is

string or list (eg. 'a' or ' ', 3 or 0)

string or list (eg. 'a' or ' ', 3 or 0)
Here the and operator performs as per following Principle

In an exprn $n \& y$ if first operand has false then return first operand n as result, otherwise return y .

X	Y	X and Y
F	F	n
F	T	n
T	F	y
T	T	y

0 and $0 \rightarrow 0$

0 and 8 $\rightarrow$ 0

5 and 0.0 →

hello and $1 \rightarrow 1$

'' and 'a' $\rightarrow$ ' '

" and " $\rightarrow$ "

 $|a\rangle$ and $|j\rangle \rightarrow |j\rangle$

$10 > 20$ and " a " $\nmid 10 \nmid 5$

(f) and (wrong)

with one you result as false.

Strings in Python :-

A string is a sequence of characters.

Strings are immutable and ordered. (Numbering of char) 13 char

first_string = 'Hello, World!' → H e l l o , w o r l d !

Print(first_string) → AM-11

len(first_string) → 13

first_string[1] → e

first_string[1] = 'o' → 0 → Error → Immutable (After Assign we can't change in string)

first_string[1:4] → string from 1 to n-1 [1 to 3]
start end cell

List 1 = [1, 2, 3]

list1[1] = 30

print(list1)

[1, 30, 3]

list1[1] = 50 → Mutable
[1, 50, 3]

String 1 = 'Hello'

String 2 = 'World'

String 3 = String 1 + String 2

print(String 3)

O/P
HelloWorld

⇒ String 1 = 'Hello'

String 2 = 20

String 3 = String 1 + String 2

print(String 3)

Error

Error

String 1 = 'Hello'

String 2 = 120

String 3 = String 1 + String 2

print(String 3)

Hello20

↓

String 1 = 'Hello'

String 2 = 20

String 3 = String 1 + String 2

print(String 3)

Hello20

change at first

String 1 = 'Hello'

String 2 = 'World'

String 3 = String 1 + ' ' + String 2
include space

print(String 3)

O/P → Hello World

Add space


```

first = 'Ram'
last = 'Varshit'
age = 30

```

Py - 3.12 Now ^{James}
 3.10 before ~~not~~
 6 - 12 ~~year~~

```

print('my first name is first & last name is last & age is age')

```

O/P my first name is first & last name is last & age is age

→ To include variable's value → Formatting (#)

```

first = 'Ram'
last = 'Varshit'
age = 30

```

```

print('# my first name is {first} & last name is {last} & age is {age}')

```

↑
Variables

my first name is Ram & last name is Varshit & age is 30.

⇒ .format

OR

String, No value, float value →
 By default convert into format to values.

```

⇒ first = 'Ram'
last = 'Varshit'
age = 30

```

```

newone = 'my first name is {} & last name is {} & age is {}'.format(first, last, age)

```

```

print(newone)

```

O/P → my first name is Ram & last name is Varshit & age is 30.

Prog to read a string & display it in Reverse order.

```

string1 = input("Enter a string:")

```

```

print("The", string1, "in reverse order is:")

```

```

length = len(string1)

```

```

for a in range(-1, (-length-1), -1):
    print(string1[a])

```

Enter a string →

The Python in reverse order is

↓
 0
 1
 2
 3
 4
 5
 6
 7
 8
 9

Slicing in Python string :- (start, stop, step)

String1 = "Ram Vashist"

R a m _ V a s h i s t
0 1 2 3 4 5 6 7 8 9 10

print(substring1) = String1[5:10] a shis

(Substring2) = String1[:10] → omit start value (Ram Vashist) Exclude 10

(Substring3) = String1[5:] → a shist (start)

(Substring4) = String1[3:] → By default all → m - vashist (Exclude 3)

substring5 = String1[-8:-3] → start stop

substring6 = String1[5::2] → start at 5 (Jump by 2)

substring7 = String1[::-1] → Reverse order print

Substr 8 = String1[5:10].upper() t shis & a v - max

Ram - Vashist → v s i t
0 1 2 3 4 5 6 7 8 9 10
A S H I S
Exclude 10
5-9

Prog to read a string & display it in the form:

first character
second character

last character
second last character

Ex:- string "toy" should print as —

t y
o o
y t

String1 = input("Enter a string:")

length = len(String1)

i = 0

for a in range(-1, (-length - 1), -1):

print(String1[i], "\t", String1[a])

i += 1

Simple run > Enter a String : Python

P y t h o n
t h o n
h o n

NOTE:- When you give a negative index, Python add length of string to get its forward index eg, for a 5-lettered string `s`, `s[-1]` will give `s[-1+5]` i.e, `s[4]` letter; for `s[-5]`, it will give you `s[-5+5]` i.e, `s[0]`

String operators —

Basic operators `+` and `*`, membership operators `in` and `not in` and comparison operators (all relational operators) for strings.

Basic operators:-

The two basic operators of string are `+` and `*`. You have used these operators as arithmetic operators before for addition and multiplication respectively. But when used with strings `+` operators

Built in fun in Python :-

Method

Description

Example

len()

Return the length of the given string.

```
>>> str1 = 'Hello World!'
>>> len(str1)
12
```

title()

Returns the string with first letter of every word in the string in uppercase & rest in lowercase.

```
>>> str1 = 'hello WORLD!'
>>> str1.title()
'Hello World!'
```

lower()

Returns the string with all uppercase letters converted to lowercase.

```
>>> str1 = 'hello WORLD!'
>>> str1.lower()
'hello world!'
```

upper()

Returns the string with all lowercase letters converted to uppercase.

```
>>> str1 = 'hello WORLD!'
>>> str1.upper()
'HELLO WORLD!'
```

count(str,
start, end)

Returns number of times substring str occurs in the given string. If we do not give start index and end index then searching starts from index 0 and ends at length of the string.

```
>>> str1 = 'Hello World!
Hello Hello'
>>> str1.count('Hello',
12, 25)
2
>>> str1.count('Hello')
3
```

find(str, start,
end)

Returns the first occurrence of index of substring str occurring in the given string. If we do not give start and end then searching starts from index 0 and ends at length of the string. If the substring is not present in the given string, then the function returns -1.

```
>>> str1 = 'Hello World!
Hello Hello'
>>> str1.find('Hello',
10, 20)
13
>>> str1.find('Hello', 15, 25)
19
>>> str1.find('Hello')
0
>>> str1.find('Hcc')
-1
```

Index(str, start, end)

Same as find() but raise an exception if the substring is not present in the given string.

```
>>> str1 = 'Hello World!
Hello Hello'
>>> str1.index('Hcc')
Value Error: substring
not found!
```

endwith()

Returns True if the given string ends with the supplied string otherwise return false.

```
>>> str1 = 'Hello world!'  
>>> str1.endswith('world!')
```

True
>>> str1.endswith('!') → True
>>> str1.endswith('ide') → False

startswith()

Returns True if the given string starts with the supplied substring otherwise return false.

```
>>> str1 = 'Hello world!'  
>>> str1.startswith('He') → True  
>>> str1.startswith('kee') → False
```

isalnum()

Returns True if characters of the given string are either alphabets or numeric. If whitespace or special symbols are part of the given string or the string is empty it returns False.

```
>>> str1 = 'Hello World'  
>>> str1.isalnum()
```

True

```
>>> str1 = 'Hello World 2'
```

```
>>> str1.isalnum()
```

True

```
>>> str1 = 'Hello world!!'
```

```
>>> str1.isalnum()
```

True False

```
>>> str1 = 'Hello World!!'
```

```
>>> str1.isalnum() ✓
```

False.

→ special char

isspace()

Returns True if the string is non empty and all characters are white space (blank, tab, newline, carriage return).

```
>>> str1 = ' \n \t \r '
```

```
>>> str1.isspace()
```

True

```
>>> str1 = 'Hello \n'
```

```
>>> str1.isspace()
```

False.

~~islower()~~

is lower

Returns True if string is non empty and has all lower alphabets or has at least one character as lowercase alphabets and rest are non alphabet characters.

```
>>> str1 = 'helloworld!'
```

```
>>> str1.islower()
```

True

```
>>> str1 = 'hello1234'
```

```
>>> str1.islower()
```

False.

```
>>> str1 = 'Hello world!'
```

```
>>> str1.islower()
```

False.

```
>>> str1 = '1234'
```

```
>>> str1.islower() → False
```


Is upper

Return True if the string is non empty and has all uppercase alphabets or has at least one character as uppercase char and rest are non-alphabet char.

```
>>> str1 = 'HELLO  
WORLD'  
>>> str1.isupper()  
True  
>>> str1 =
```

List :- The data type list is an ordered sequence that is mutable & made up of one or more elements.

A list can have elements of different data types, such as integer, float, string, tuple or even another list.

A list is very useful to group together elements of mixed data types.

Elements of a list are enclosed in sequence brackets and are separated by a comma.

List indices also start from 0.

list 1 is an example of mixed data types.

```
>>> list1 = [20, 5.5, 'Hello']
```

```
>>> print(list1)
```

```
[20, 5.5, 'Hello']
```

list 2 is an example of nested list.

```
>>> list2 = [['Ram', 1989], ['Ravinder', 1990], ['Shyam',  
1991]]
```

```
>>> print(list2)
```

```
 [['Ram', 1989], ['Ravinder', 1990], ['Shyam', 1991]]
```

Initializes a list list1

```
>>> list1 = [20, 5.5, 'Hello']
```

```
>>> list1[0] # Return first element of list 1.
```

20

>>> list1[2] # return third element of list1
Hello

hello-good

>>> list1[5] → Error
>>> list1[-1] → Hello
>>> list1[n-1] → Hello
>>> list1[-n] → 20
>>> list1[2] = 'good'

let $n=5$ 0 to $n-1$

(last element) 1 2 3 4 5
 0 1 2 3 4 → index
 0 $n-1$
 -n -1

Various operations on list:-

1) Concatenation :- +

list1 is a list of the first five odd integers.

>>> list1 = [1, 3, 5, 7, 9]

list2 is a list of the first five even integers.

>>> list2 = [2, 4, 6, 8, 10]

elements of list1 followed by list2

>>> list1 + list2

[1, 3, 5, 7, 9, 2, 4, 6, 8, 10]

2) Repetition :- *

>>> list1 = ['hello']

elements of list1 repeated 5 times.

>>> list1 * 5

['hello', 'hello', 'hello', 'hello', 'hello']

3) Membership :- in

>>> list1 = ['Varun', 'Ravinder', 'Amsit']

>>> 'Amsit' in list1 → True

Amsit not in list1 → F

>>> 'Nitin' in list1 → false

Nitin not in list1 → F

4.) Slicing:-

[start End Index jump] ^(stop-1)

>>> list1 = ['varun', 'Ravinder', 'Amit', 'Nitin', 'Rahul', 'Prayansh']

>>> list1[2:4]

o/p → ['Amit', 'Nitin']

a[start:stop] → # items start through stop-1. (→ Rahul)

a[start:] → # items start through the rest of the array.

a[:stop] → # items from the beginning through stop-1

a[:] → # a copy of whole array.

a[-1] → # last item in the array.

a[-2:] → # last two items in the array.

a[:-2] → # Everything except the last two items.

a[::-1] → # all items in the array, reversed.

a[1::-1] →

a[:-3:-1] →

a[-3::-1] →

N R A N R P

list1[2:4:2] → Amit

start stop jump

Modules :- A module is basically a Python file consisting of Python executable code, functions, statements, definitions, global variables etc.

Modules are used to break down large program into small manageable and organized sections for providing reusability of code like functions.

A Python file named "usermodule.py" is called a module and the name of the module is "usermodule".

The functions and definitions inside a module can be imported in the program using the "import" keyword.

The followings are the three module approaches to use the existing module in the program.

- 1) `from module import *` : import all functions from module.
- 2) `import module` : import everything and import the module itself as a unit.
- 3) `from module import function1, function2 -- :` import specified functions from module.

There is a small difference between the first two approaches. The first approach imports everything and the second approach imports the modules as a unit.

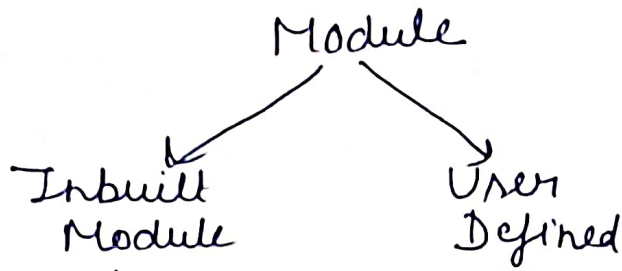
Though the first approach, the user needs to write the name of the module and function, both of them are separated by dot.

In second approach the function name can be directly written without specifying the name of the module.

The first approach requires programmers to use the longer, qualified names for modules.

There are two types of Modules containing ^{Python} specific functions such as Python - inbuilt modules and user defined modules.

Provide Proper Organization.
Code usability
Namespace Management.
Encapsulation.



Inbuilt modules available in Python are related to specific function of the Particular Module

Helps the user to create own module for effective programming through Manageable code and reusability of code.

Inbuilt Module in Python:-

Python has a lot of predefined modules containing specific functions that can be directly used in programming. Some of the commonly used modules include sys, math, collections, array, re, statistics, datetime, random, clock etc.

1.) In Built Module:-

- 198
- 1.) The math Module
 - 2.) The Random Module
 - 3.) The statistics Module
 - 4.) The Array Module
 - 5.) The string Module

- 1.) Accessing string Elements
- 2.) Case Conversion Functions
- 3.) Alignment and Indentation Functions.
- 4.) Other functions for string.

- 6.) The "re" Module
- 7.) The time Module
- 8.) The Datetime Module
- 9.) The "os" Module

2-) User-Defined Module:-

Creating a Module

Importing the User-Defined Module

1.) The Math Module:-

The math module includes basic mathematical, logarithmic and trigonometric functions.

The different mathematical functions include `fabs()`, `Power()`, `factorial()`, `floor()`, `ceil()` etc. Logarithmic functions include `log()`, `log10()` etc. and trigonometric functions include `sin()`, `cos()`, `tan()` etc. It is important to note that the user needs to import math module before using the functions available from this module.

Importing math module

```
import math
```

The `floor()` → `fn` returns the lowest integer of the number.

```
print("Floor of 28.6 is:", math.floor(28.6))
```

Floor of 28.6 is 28

The `ceil()` → `fn` returns the highest integer of the number.

```
print("ceiling of 28.6 is:", math.ceil(28.6))
```

ceiling of 28.6 is 29

The `factorial()` → `fn` returns the factorial of a number.

```
print("Factorial of 9 is:", math.factorial(9))
```

Factorial of 9 is 362880

The `pow()` → function raises one number to a power of another.

```
print("Number 8 to the Power 4 is:", math.pow(8,4))
```

no 8 to the power 4 is: 4096.0

The `sqrt()` → function returns the square root of a number.

`print("square root of 12 is :", math.sqrt(12))` → 3.4641016151377544

The `fabs()` → Returns the absolute value of a no.

`print("Float absolute value of -186.46 is :", math.fabs(-186.46))` → float absolute value of -186.46 is : 186.46

Use of Trigonometric functions from `math` module.

The `sin()` → `sin` returns the `sin` of specified value in radians.

`print("sin of 180 is :", math.sin(180))` → $\sin$ of 180 is → -0.801152635733836

The `cos()` → `cos` returns the `cosine` of specified value in radians.

`print("cosine of 180 is :", math.cos(180))` → $\cos$ of 180 is → -0.5984600690578582

The `tan()` → `tan` returns the `tan` of specified value in radians.

`print("Tan of 180 is :", math.tan(180))` → $\tan$ of 180 is → 1.3386902103511544

Use of Logarithmic functions from `math` module.

the `log()` :- The function returns natural logarithm of no.; $\log(n) = \log_e x$.

`print("log of 2000 with base e is :", math.log(2000))`
Log of 2000 with base 10 is → 3.3010299956639813
7.600902459542082

The `log10()` → `log` Returns logarithm of no $\log(n) = \log_{10} x$

`print("log of 2000 with base 10 is :", math.log10(2000))` → 3.3010299956639813

The `exp()` → `exp` returns result of `e` raised to power `x`; $\exp(x) = e^x$

`print("Exponential of 256 is :", math.exp(256))`

Exponential of 256 is : 1.5114276650041035e+111

2.) The Random Module :-

The random module in python contains standard functions related to pseudorandom numbers.

The `random()`, `randrange()` and `seed()` functions are important.

Program to use `random()` and `randrange()` functions from random module.

The `random()` → Returns floating point number x in the range $0 \leq x < 1$.

```
print("The random number generated is", random())
```

```
print("The new random number generated is:", random())
```

3.) Randrange

4) datetime :- Import datetime

```
now = datetime.datetime.now()
```

```
print(now)
```

In this example we import the `datetime` module and use its `datetime()` constructor to create a new `datetime` object representing the current date and time. The resulting object is then printed to the console.

5) File input and Output Module

function :- A function is a group of statements that exist within a program for the purpose of performing a specific task.

Problem has been divided into smaller tasks / sub-prob. each of which is performed by a specific function. This approach to problem solving is called divide and conquer approach, or modular approach or stepwise refinement method.

Types of function :-

- ① Built-in function
- ② User-defined function.

① Built-in function :- There are predefined library functions that are already available in Python.

function is used to separate code.

There are 4 types of arg that we can provide in a function.
Default Arg Keyword Arg variable length ^{Arg} keyword Required Arg.

Ex: `def average(a, b):` while creating a fn. This way fn assume a default value even if a val. not provided in the fn cell for that arg.
`print("the average is ", (a+b)/2) → x`
- `average(4, 8) → 5.0`

1)

```
def average (a=9, b=1):
```

```
    print
```

```
    average() → 5
```

```
    average(1, 5) → 3
```

```
    average(5) → 3    b/c val. by default 1.
```

```
    average(b=9) → By def a=9, b=1 → avg → 4.0
```

```
def name (fname, mname = "John", lname = "Whetson");
```

```
    print("Hello;", fname, mname, lname)
```

```
    name("Amy")
```

```
O/P → Hello, Amy John Whetson
```

```
Average (b=9, a=21)
```

2) Keyword Arguments:—
We can provide argument with key = value, this way the interpreter recognizes the arguments by the parameter name. Hence the order in which the arguments are passed does not matter.

```
avg average(b=9, a=21) → O/P 15.0
```


3) Required Arguments :-

In this case we don't pass the arguments with a key-value syntax, then it is necessary to pass the arguments in the correct positional order and the number of arguments passed should match with actual function definition.

Ex when no. of arguments passed does not match to the actual function definition.

```
def name(fname, mname, lname):  
    print("Hello", fname, mname, lname)  
name("Peter", "Ouel")
```

O/P - Error → Missing a required Arg.

```
def average(a, b, c=1):  
    print("The average is ", (a+b+c)/2)  
average(5, 6)
```

O/P → The average is 6.0

4) Variable length Arguments :-

Sometimes we may need to pass more arguments than those defined in the actual function. This can be done using variable length argument.

There are 2 ways to achieve this -

1-) Arbitrary Argument :-

While creating a function pass * before the parameter name while defining the function. The function access the arguments by processing them in the form of tuple.

Ex

```
def name(*name)
```

```
    print("Hello, ", name[0], name[1], name[2])
```

```
    name(*James, "Watson", "Henry")
```

→ all

Hello, James
Watson
Henry

```
def average (*numbers):
    sum = 0
    for i in numbers:
        sum = sum + i
    print("Average is:", sum / len(numbers))

average(5, 6) → O/P Average is 5.5
```

Program using *arg & **kwargs

```
def bakery(item, *args, **kwargs):
```

```
    print("We are out of {item}")
```

```
    print("What all items available?")
```

```
    print("Items available are:")
```

```
    for arg in args: [ Loop start here from arg to args ]
```

```
        print(arg)
```

```
    print("_" * 30)
```

→ for break the statement of output

```
    for kw in kwargs:
```

→ [Loop initiate from kw to kwargs]

```
        print(kw, ': ', kwargs[kw]) [ #xh cell[kw] in kwargs ]
```

```
    bakery("Cake", "Bread", "Biscuit", "Laddu", "Icecream",
```

```
        shop_name="Bread factory", location="TVM")
```

Output :- we are out of cake
 What all items available? [print arg]

→ Items available are:

Bread
 Biscuit
 Laddu
 Ice cream

Code
 ["_" * 30]

Shop Name : Bread factory
 Location : TVM

[print kwargs]